

코드 한 줄 몰라도 괜찮아

비개발자가 바이브코딩으로 감사일기 앱을 출시하기까지



아이디어만 있으면 누구나 앱 개발자가 될 수 있다

엄세라 지음

저자 소개

안녕하세요, 엄마의 세컨드 라이프를 준비하는 엄세라입니다.

저는 개발자가 아니에요. 코딩을 배운 적도 없고, 컴퓨터 공학을 전공하지도 않았어요. 그냥 평범한 콘텐츠 크리에이터이자 블로거예요.

그런 제가 어느 날 갑자기 앱을 만들어서 Google Play Store에 출시했어요. 그것도 코드 한 줄 직접 짜지 않고요.

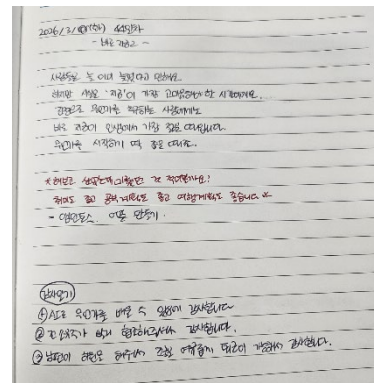
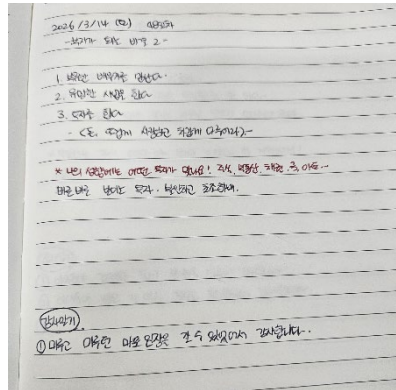
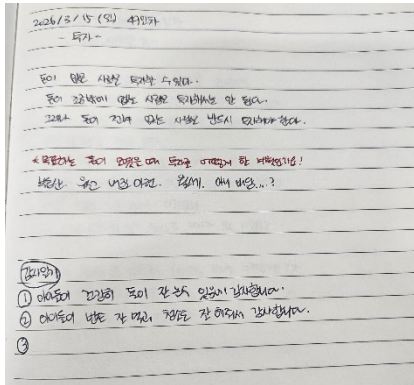
이 전자책은 제가 실제로 겪은 이야기예요. 잘 된 것만 쓴 게 아니라, 에러가 났던 순간, 모르는 용어가 나와서 막혔던 순간, 포기하고 싶었던 순간까지 솔직하게 담았어요.

왜냐하면, 제가 처음 시작할 때 이런 솔직한 이야기가 담긴 가이드가 없어서 너무 힘들었거든요. 이 책이 여러분의 시작을 조금이라도 쉽게 만들어드리길 바라요.

들어가며 — 나는 왜 앱을 만들기로 했을까

감사일기를 쓰다가 생긴 아이디어

저는 매일 감사일기를 썼었어요. 부자 필사노트를 쓰면서 오늘 감사한 일 3가지를 적는 거예요. 처음엔 그냥 노트에 썼어요. 그러다 뭔가 아쉽게 느껴지더라고요. 손도 아팠고요.



▲ 직접 손으로 쓰던 감사일기 노트

어느 날 문득 이런 생각이 들었어요.

"매일 감사일기를 쓰는 사람들이 나 말고도 많을 텐데, 이것 위한 앱이 있으면 좋겠다. 내가 만들면 어떨까?"

그런데 바로 현실의 벽에 부딪혔어요. '나는 코딩을 모르는데?'

그때부터 방법을 찾기 시작했어요. 유튜브도 보고, 앱인토스도 해봤는데... 초반에만 잘 알아듣겠고, 그 이후부터는 너무 어렵더라고요. 시간 내서 배우면 할 수 있겠지만, 워킹맘에겐 그럴 시간이 허락되지 않았거든요. 저는 그냥 아이디어를 현실로 만들고 싶었을 뿐이었어요.

바이브코딩을 만나다

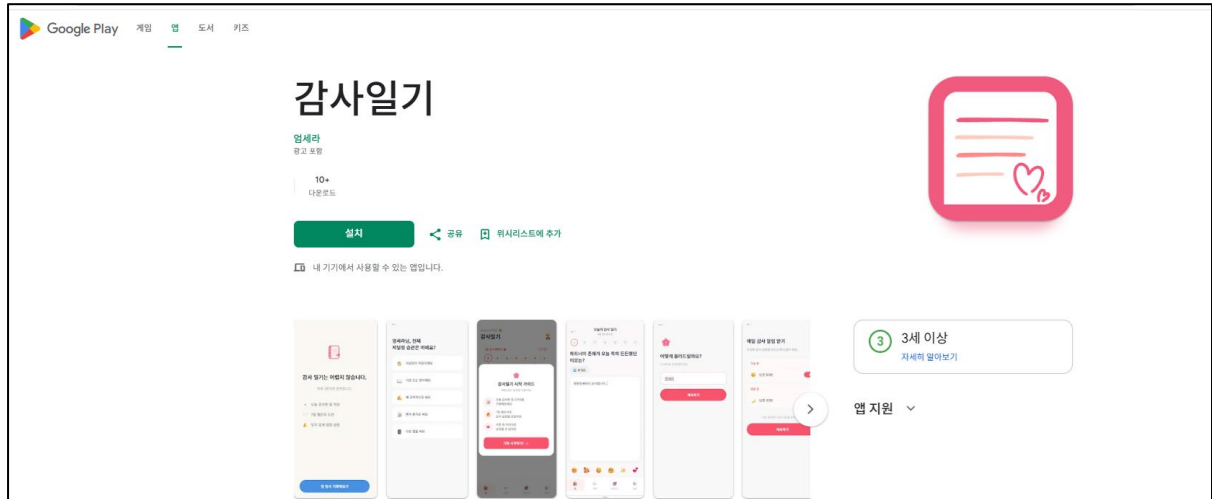
그러던 중 바이브코딩이라는 개념을 발견했어요. 그런데 때마침 스레드라는 SNS에서 AI로 앱을 출시했다는 어떤 분의 계정을 알게 된 거예요. 게다가 저와 같은 엄마라고 하셨어요. '오? 나도 할 수 있는건가?' 싶은 자신감에 계속 대화를 나누는데 그 과정을 전자책으로도 만드셨다고 하더라고요. 전자책을 읽어보고 저도 한번 '해볼까?'하게 되었죠. 사람들은 나보다 더 뛰어난 전문가보다, 나와 비슷한 사람이 한 단계 앞서 나간 것에 더 큰 동질감을 느끼고 동기부여가 되잖아요.

바이브코딩은 AI에게 말하듯 대화하면서 앱을 만들어가는 방식이에요. '이런 기능 만들어줘',

'이 색깔로 바꿔줘' 하면 AI가 직접 코드를 짜주는 거예요.

처음엔 반신반의했어요. '진짜 이게 되나?' 싶었죠. 그런데 해보니까 진짜 되더라고요.

물론 쉽지만은 않았어요. 예러도 엄청 많이 났고, 용어를 몰라서 헤맨 적도 많았어요. 하지만 포기하지 않고 하나씩 해결해나갔고, 결국 Play Store에 앱을 출시했어요.



▲ 구글 플레이 스토어에 출시된 엄세라의 감사일기 어플

이 책이 도움이 되는 분

- 앱 아이디어는 있는데 코딩을 모르는 분
- 개발자에게 외주를 맡기기엔 비용이 부담스러운 분
- 사이드 프로젝트로 앱을 만들어보고 싶은 분
- 바이브코딩이 뭔지 궁금한 분
- AI를 활용해서 무언가를 만들어보고 싶은 분

준비되셨나요? 같이 시작해봐요! ✿

목차

- 1장. 바이브코딩이란 무엇인가
- 2장. 아이디어를 앱으로 — React로 감사일기 만들기
- 3장. 웹앱을 진짜 앱으로 — Capacitor 변환하기
- 4장. 광고로 수익 만들기 — AdMob 연동
- 5장. 앱을 더 풍부하게 — 이미지 저장과 디자인
- 6장. 드디어 출시! — Google Play Store 등록하기
- 7장. 출시 이후 — 앱을 성장시키는 방법
- 8장. 첫 출시 거부 – 진짜 시작은 여기서부터

부록. 자주 묻는 질문 (FAQ)

1 장. 바이브코딩이란 무엇인가

코딩 없이 앱을 만든다고?

'바이브코딩(Vibe Coding)'이라는 말, 들어보셨나요? 아직 생소한 분들이 많을 거예요. 바이브코딩은 AI에게 자연어로 대화하듯 요청해서 코드를 만들어나가는 개발 방식이에요.

기존의 코딩은 그랬어요. 프로그래밍 언어를 배우고, 문법을 익히고, 수백 줄의 코드를 직접 짜야 했죠. 최소 몇 달은 배워야 간단한 앱 하나 만들 수 있었어요.

바이브코딩은 달라요. 이렇게 하면 돼요.

"핑크색 배경에 감사일기라는 제목을 크게 쓰고, 아래에 오늘의 감사한 일을 입력하는 텍스트 박스를 만들어줘"

이렇게 말하면 AI가 그에 맞는 코드를 짜줘요. 마치 개발자 친구에게 말하는 것처럼요.

왜 지금 바이브코딩인가

2024년부터 AI 기술이 폭발적으로 발전했어요. Claude, Gemini, ChatGPT 같은 AI들이 코드를 정말 잘 짜게 됐거든요. 덕분에 비개발자도 아이디어만 있으면 실제로 동작하는 앱을 만들 수 있게 됐어요.

물론 아직 한계가 있어요. 아주 복잡한 기능이나 대규모 서비스를 만들려면 전문 개발자가 필요해요. 하지만 개인 앱, 사이드 프로젝트, MVP(최소 기능 제품) 수준이라면 충분히 바이브코딩으로 만들 수 있어요.

바이브코딩의 핵심 원칙 5 가지

1. 구체적으로 말하기

AI에게 요청할 때는 최대한 구체적으로 말해야 해요. 막연하게 '예쁘게 만들어줘'가 아니라 '핑크색(#F9546E) 배경에 흰색 텍스트로 제목을 넣어줘'처럼요.

💡 색상은 hex 코드로, 크기는 픽셀로, 위치는 '위쪽', '오른쪽' 처럼 구체적으로 말해주세요.

2. 한 번에 하나씩

여러 기능을 한꺼번에 요청하면 AI가 혼동할 수 있고, 에러가 생겼을 때 어디서 문제가 생겼는지 파악하기 어려워요. 하나씩 만들고, 확인하고, 다음 단계로 넘어가세요.

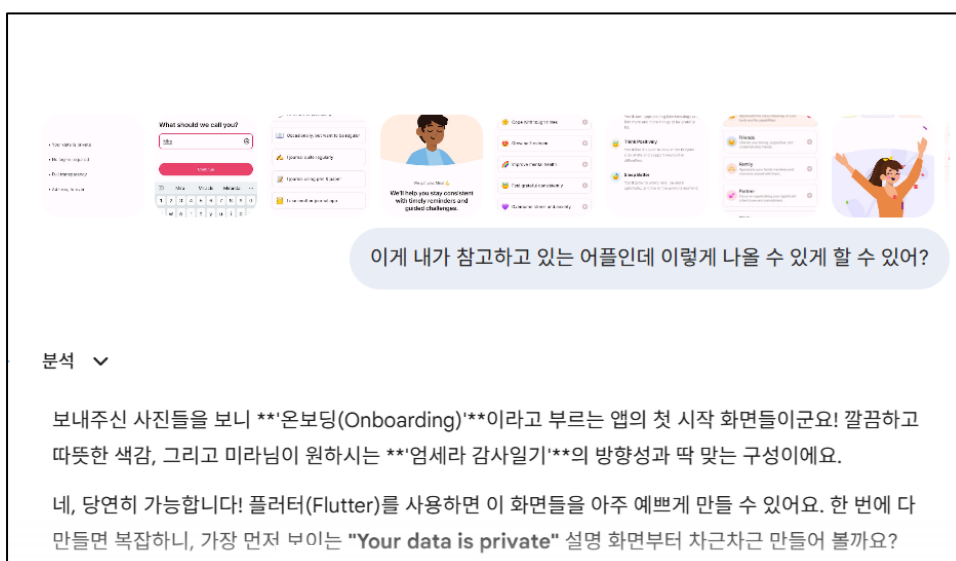
3. 에러를 두려워하지 않기

에러는 당연히 납니다. 에러가 안 나는 개발은 없어요. 중요한 건 에러 메시지를 그대로 AI에게 복사해서 붙여넣으면 대부분 해결된다는 거예요.

💡 에러 메시지를 보고 겁먹지 마세요. 빨간 글씨로 나온 내용을 그대로 복사해서 AI에게 보여주면 해결책을 알려줘요.

4. 스크린샷 활용하기

내가 원하는 디자인이 있다면 참고 이미지를 AI에게 보여주세요. '이런 느낌으로 만들어줘'라고 하면 훨씬 정확하게 만들어줘요. 저는 마음에 드는 앱 스크린샷을 찍어서 보여줬어요.



▲ Gemini 에게 참고 이미지를 보여주는 모습

5. 저장 습관 들이기

코드를 수정할 때마다 저장(Ctrl+S)하는 습관을 들이세요. 그리고 중요한 변경사항이 있을 때는 백업을 해두는 게 좋아요. 저는 한 번 실수로 코드를 날린 적이 있었는데, 정말 멘붕이었거든요.

어떤 AI 를 써야 할까?

여러 AI가 있지만, 저는 Claude를 주로 사용했어요. 코드를 잘 짜는 것은 물론이고, 예러 해결도 잘 해줬어요. 무엇보다 대화하듯 물어볼 수 있어서 편했어요. 사실 Gemini 유료버전을 쓰다가, Claude가 무료 버전으로 해봤는데도, 너무 예쁘게 나와서 갈아탔어요. 이미 Gemini랑 ChatGPT 유료구독을 하고 있었지만, Claude도 너무 예뻐서 바로 구독.. 구독 도파민..

필요한 도구들

바이브코딩을 시작하기 전에 몇 가지 도구를 설치해야 해요. 처음엔 낯설 수 있지만, 설치 자체는 어렵지 않아요.

- VS Code — 코드를 편집하는 도구. 그냥 메모장이라고 생각하면 돼요.
- Node.js — 앱을 실행하기 위한 프로그램. 설치만 하면 돼요.
- Android Studio — Android 앱을 테스트하고 빌드하는 도구.
- Claude 또는 ChatGPT — AI 도우미.

💡 이 도구들 설치에 막히면 AI에게 '맥북(또는 윈도우)에 VS Code 설치하는 방법 알려줘'라고 물어보세요. 단계별로 설명해줘요.

2 장. 아이디어를 앱으로 — React 로 감사일기 만들기

감사일기 앱, 어떻게 기획했나

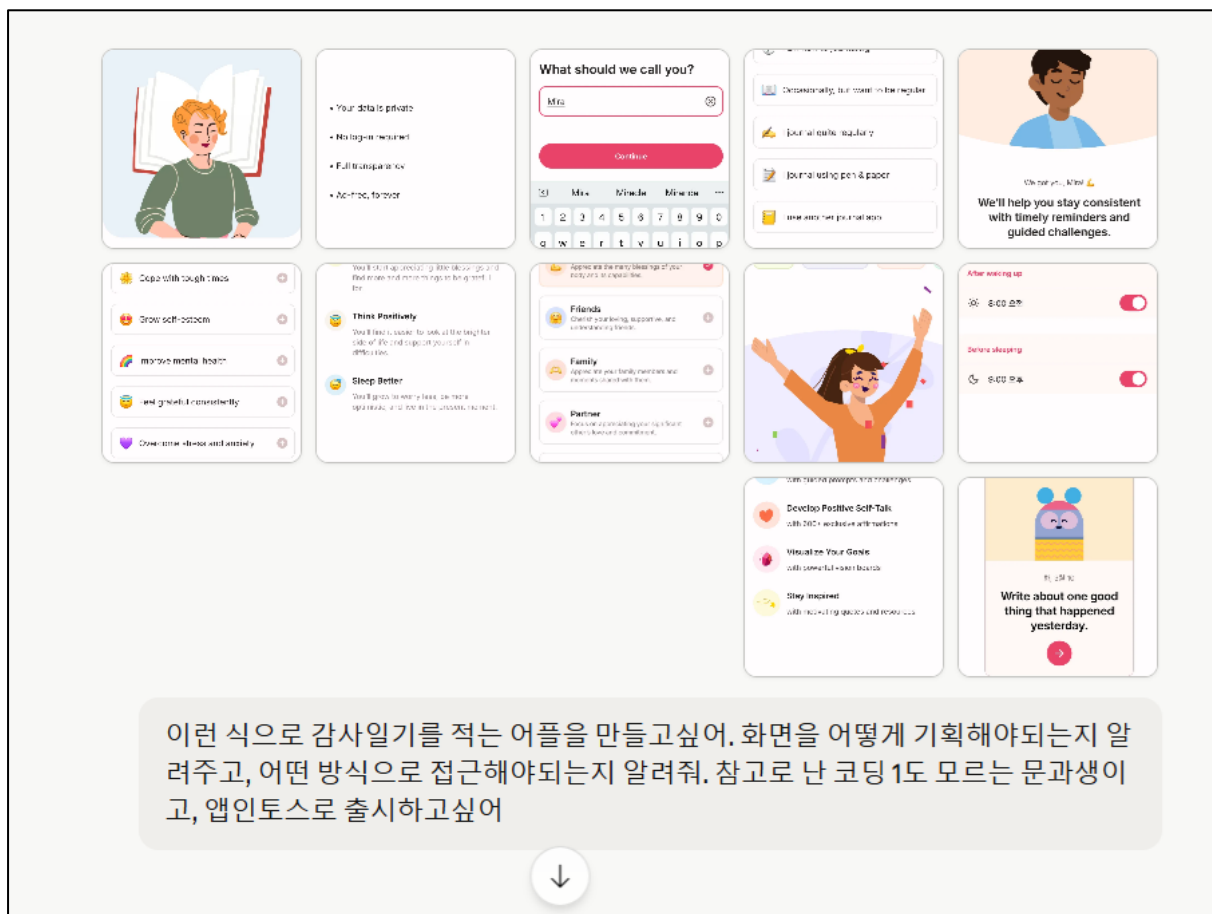
앱을 만들기 전에 먼저 기획이 필요해요. 어떤 기능을 넣을 건지, 화면이 몇 개인지, 어떤 흐름으로 사용자가 앱을 쓸 건지를 생각해야 해요.

저는 Claude에게 이렇게 물어봤어요.

"감사일기 앱을 만들고 싶어. 매일 감사한 일 3가지를 쓰는 앱인데, 어떤 기능들이 있으면 좋을까?"

Claude가 다양한 아이디어를 제안해줬어요. 그중에서 제가 원하는 것들을 골랐죠.

▼ Claude와 함께 감사일기 앱 기능을 기획하는 대화



감사일기 앱의 핵심 기능

- 온보딩 플로우 — 처음 앱을 실행했을 때 닉네임 입력, 감사 이유 선택, 주제 선택
- 7 일 챌린지 스트릭 — 연속으로 몇 일 썼는지 시각적으로 보여주기
- 오늘의 감사 일기 작성 — 프롬프트 질문과 함께 일기 쓰기
- 감사 명언 — 매일 다른 명언 보여주기
- 비전보드 — 내가 원하는 것들을 사진으로 모아두기
- 알림 설정 — 아침, 저녁 원하는 시간에 알림 받기

React 가 뭔지 몰라도 괜찮아요

React는 웹 앱을 만드는 자바스크립트 라이브러리예요. 이 설명이 무슨 말인지 몰라도 괜찮아요. 저도 처음엔 몰랐거든요.

중요한 건 React로 만들면 나중에 Capacitor라는 도구를 이용해서 Android/iOS 앱으로 변환할 수 있다는 거예요. 그래서 React를 선택했어요.

Claude에게 이렇게 말했어요.

"React로 감사일기 앱을 만들어줘. 첫 화면은 온보딩 화면이고, 닉네임을 입력 받고 싶어. 배경색은 연한 핑크색이고, 따뜻한 느낌이었으면 좋겠어."

그러면 Claude가 코드를 짜줘요. 그 코드를 VS Code에 붙여넣고 실행하면 앱이 브라우저에서 열려요.

화면 하나씩 만들어가기

저는 화면을 하나씩 만들어갔어요. 온보딩 화면이 완성되면, 다음 화면으로 넘어가는 식으로요.

1) 온보딩 화면

첫 화면은 사용자를 반갑게 맞이하는 화면이에요. 닉네임을 입력받고, 왜 감사일기를 쓰고 싶은지 이유를 선택하게 했어요.

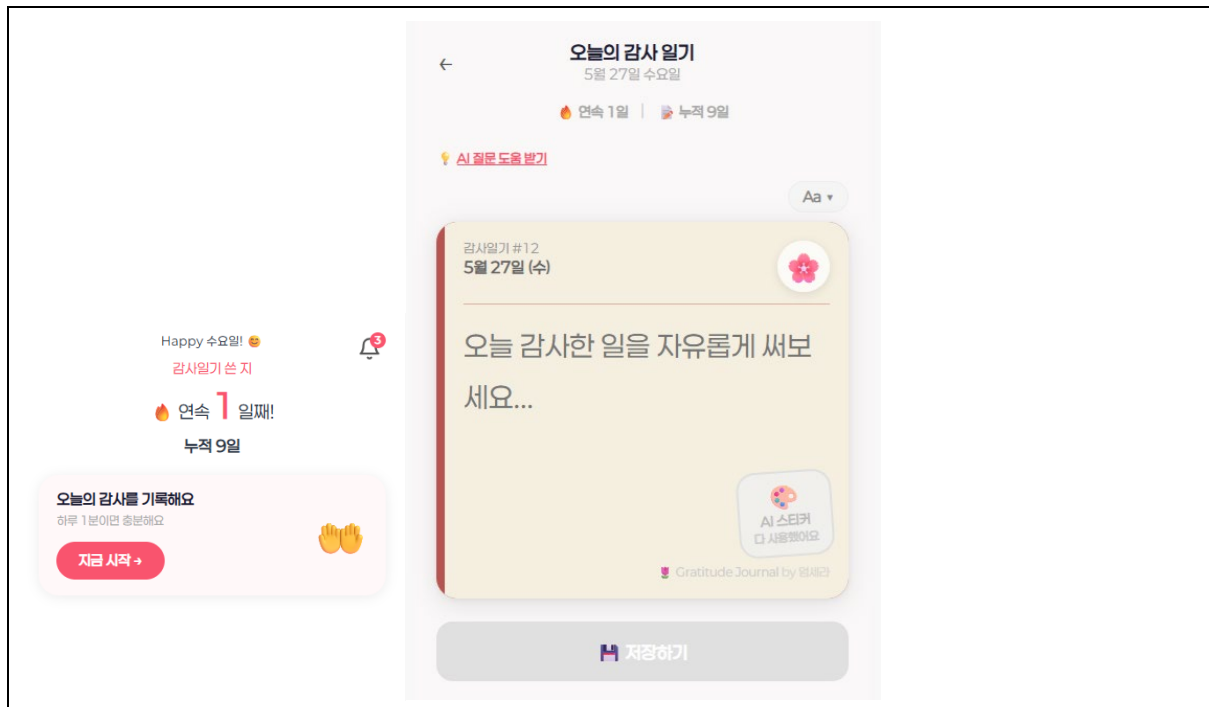
💡 온보딩은 사용자의 첫인상을 결정해요. 따뜻하고 친근한 문구를 넣어주세요.

2) 메인 홈 화면

닉네임 입력, 기록 습관, 감사일기 주제, 알림 설정하면 보이는 메인 화면이에요. 7일 스트릭, 오늘의 명언, 일기 쓰기 버튼을 배치했어요. 시각적으로 예쁘게 만들고 싶어서 Claude에게 여러 번 수정을 요청했어요.

3) 일기 작성 화면

감사 프롬프트 질문이 뜨고, 이모지를 선택하고, 텍스트를 입력하는 화면이에요. 여기서 저장 버튼을 누르면 광고가 뜨도록 나중에 연동했어요.



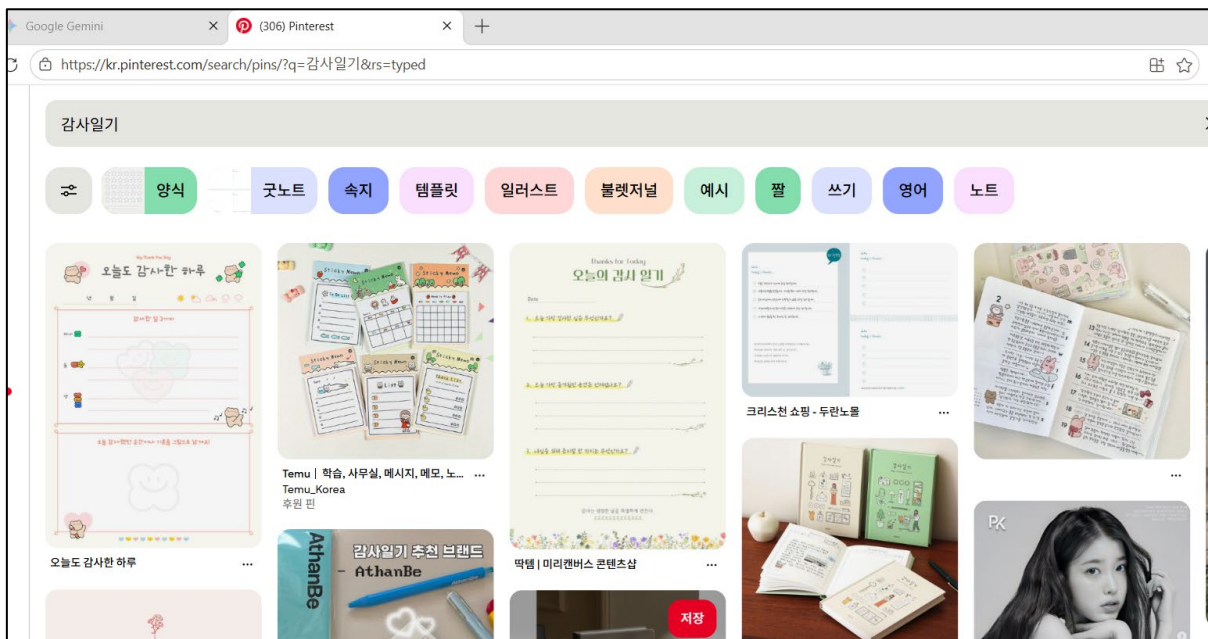
▲ 완성된 감사일기 앱 - 홈 화면과 온보딩 화면

디자인은 어떻게 했나

저는 디자인을 전혀 모르지만, 괜찮아요. Claude에게 참고 이미지를 보여주거나 색상 코드를 알려주면 비슷하게 만들어줘요.

저는 아이콘을 먼저 만들고, 그 아이콘의 색상(#F9546E, 핑크 레드)을 앱 전체 컬러로 통일했어요. 일관된 컬러 팔레트가 있으면 전체적으로 훨씬 깔끔해 보여요.

💡 앱 디자인이 어렵다면 dribbble.com이나 pinterest.com에서 참고 디자인을 찾아서 AI에게 보여주세요.



▲ 핀터레스트로 참고 디자인 찾는 모습

▲ VS CODE 화면

터미널, 이것만 알면 돼요

처음 터미널을 접하면 어렵게 느껴질 수 있어요. 하지만 실제로 쓰는 명령어는 몇 가지 안 돼요.

- `npm start` — 개발 서버 실행 (브라우저에서 앱 미리보기)
- `npm run build` — 앱 빌드 (최종 파일 생성)
- `npx cap sync` — Capacitor 에 최신 빌드 반영
- `npx cap open android` — Android Studio 열기
- `Ctrl+C` — 실행 중인 서버 종료

💡 터미널에서 명령어를 치면 뭔가 주르륵 나와요. *'webpack compiled with X warnings'* 같은 노란 경고는 무시해도 돼요. 빨간 에러만 신경 쓰면 돼요.

개발 서버 vs 빌드, 뭐가 다를까?

처음에 저도 헷갈렸는데, 쉽게 설명하면 이래요.

- 개발 서버 (`npm start`) — 코드 수정한 걸 브라우저에서 미리 보는 것. 개발 중에 써요.
- 빌드 (`npm run build`) — 앱 완성본 만드는 것. Play Store 에 올릴 때 써요.

💡 개발 중에는 `npm start`로 변경사항을 실시간으로 확인하고, 다 완성되면 `npm run build` → `npx cap sync` 순서로 진행하세요.

Android Studio 설정

Android 앱을 빌드하려면 Android Studio가 필요해요. Android Studio는 Google에서 만든 Android 개발 도구예요. 무료로 다운로드할 수 있어요.

설치 후 에뮬레이터(가상 핸드폰)를 설정하면 실제 핸드폰 없이도 앱을 테스트할 수 있어요.

💡 *에뮬레이터는 처음 켤 때 시간이 꽤 걸려요. 인내심을 갖고 기다리세요. 부팅 완료 표시가 뜰 때까지 기다려야 해요.*

이때 나타나는 에러들

솔직히 이 단계에서 에러가 가장 많이 났어요. 제가 경험한 에러들과 해결 방법을 공유할게요.

경로 에러 (cd 명령어 실패)

터미널에서 'cd gratitude-app'을 쳤는데 '경로가 존재하지 않습니다'라는 에러가 뜰 수 있어요. 이미 해당 폴더 안에 있는데 또 들어가려고 해서 그래요.

터미널 프롬프트를 확인해보세요. 'C:\Users\User\Downloads\gratitude-app\gratitude-app>' 이렇게 이미 들어가 있으면 그냥 명령어를 치면 돼요.

JAVA_HOME 에러

Android 빌드를 위해 Java가 필요한데, 설치가 안 되어 있거나 경로 설정이 안 된 경우에 나타나요. 에러 메시지를 Claude에게 보여주면 운영체제에 맞는 해결 방법을 알려줘요.

에뮬레이터 연결 실패

에뮬레이터가 켜져 있는데도 앱이 설치가 안 되는 경우예요. 에뮬레이터를 완전히 끄고 Cold Boot으로 다시 시작해보세요.

💡 *Cold Boot은 Android Studio의 Device Manager에서 에뮬레이터 옆 메뉴(...)를 클릭하면 찾을 수 있어요.*

Gradle 빌드 실패

Gradle은 Android 앱을 빌드하는 도구예요. 빌드가 실패하면 빨간 에러 메시지가 나와요. 당황하지 말고 에러 메시지를 Claude에게 보여주세요.

핸드폰에서 직접 테스트하는 방법

에뮬레이터 말고 실제 핸드폰에서 테스트하는 방법도 있어요. PC와 핸드폰이 같은 와이파이
에 연결되어 있으면 가능해요.

- 1 단계: npm start 실행
- 2 단계: 터미널에서 'On Your Network: http://192.168.xxx.xxx:3000' 주소 확인
- 3 단계: 핸드폰 브라우저에서 그 주소 입력

💡 회사 유선 네트워크라면 핸드폰 핫스팟을 켜고 PC를 핫스팟에 연결한 다음 `ipconfig`
명령어로 IP 주소를 확인해보세요. 하지만 저는 실패하고 그냥 웹앱이나 *Android Studio*
에서만 확인해봤어요. ππ

4 장. 광고로 수익 만들기 — AdMob 연동

무료 앱으로도 돈을 벌 수 있어요

앱을 무료로 출시하면서도 수익을 낼 수 있는 방법이 있어요. 바로 광고예요. Google AdMob은 앱 안에 광고를 넣어서 수익을 창출하는 서비스예요.

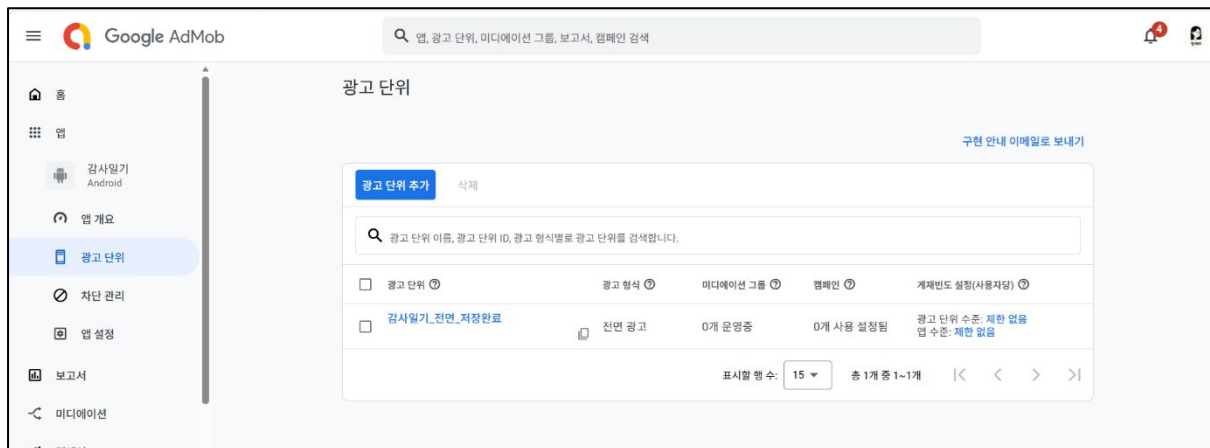
사용자가 광고를 볼 때마다 약간의 수익이 발생해요. 처음엔 금액이 작을 수 있지만, 사용자가 늘어날수록 수익도 늘어나요.

광고 유형 이해하기

- 배너 광고 — 화면 하단에 작게 고정되는 광고. 항상 노출돼요.
- 전면 광고 — 화면 전체를 가리는 광고. 화면 전환 시 잠깐 보여요.
- 보상형 광고 — 사용자가 광고를 보면 보상을 주는 방식.

저는 일기를 저장할 때 전면 광고가 뜨도록 설정하려고 했어요. 사용자가 일기를 다 쓰고 저장 버튼을 누르는 순간이 자연스럽게 광고를 볼 수 있는 타이밍이에요.

▼ Admob 광고 유형 설정화면



AdMob 계정 만들기

admob.google.com에 접속해서 Google 계정으로 로그인하면 돼요. 앱을 등록하면 앱 ID와 광고 단위 ID를 받을 수 있어요.

💡 AdMob 계정 승인에 며칠이 걸릴 수 있어요. 미리 신청해두는 게 좋아요.

반드시 테스트 광고로 먼저 확인하세요

처음 설정할 때는 반드시 Google에서 제공하는 테스트 광고 ID를 사용해야 해요. 실제 광고 ID로 테스트하면 Google 정책 위반이 될 수 있거든요.

테스트 광고 ID: ca-app-pub-3940256099942544/1033173712

💡 광고가 뜨는 데 처음엔 시간이 걸릴 수 있어요. 저는 처음에 광고가 안 뜨는 줄 알고 몇 시간을 씨름했는데, 사실 조금만 기다리면 됐던 거였어요.

하지만 결국 애드몹 광고는 코드엔 심어놓고 비활성화로 해두었어요. 감성에 안 맞는 것 같아서요.

5 장. 앱을 더 풍부하게 — 이미지 저장과 디자인

인스타 공유용 이미지 저장 기능 만들기

앱을 쓰면서 이런 생각이 들었어요. '오늘 쓴 감사일기를 예쁜 이미지로 저장해서 인스타그램에 올리면 어떨까?' 이게 바이트코딩의 장점이에요. 새 기능을 추가하고 싶으면 그냥 AI에게 말하면 돼요.

html2canvas라는 라이브러리를 사용하면 웹 화면을 이미지로 저장할 수 있어요. Claude에게 '감사일기 오늘 일기를 인스타 공유용 이미지로 저장하는 기능 만들어줘'라고 하면 알아서 코드를 짜줘요.

라이브러리 설치

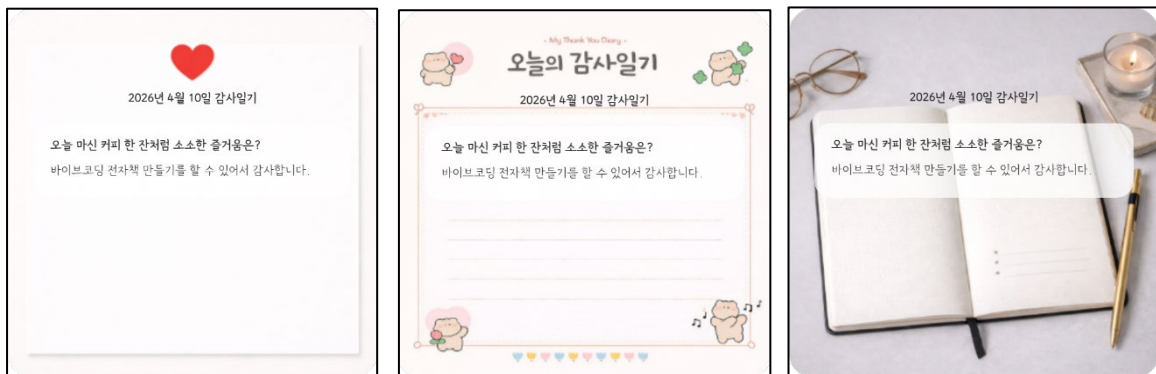
```
npm install html2canvas
```

배경 이미지 선택 기능

단순히 흰 배경에 텍스트만 저장하면 재미없잖아요. 저는 ChatGPT로 예쁜 배경 이미지 3가지를 만들어서 사용자가 선택할 수 있게 했어요.

- ChatGPT 로 앱 컨셉에 맞는 배경 이미지 생성 (노트, 꽃, 따뜻한 느낌 등)
- src 폴더에 bg1.png, bg2.png, bg3.png 로 저장
- 배경 선택 모달 팝업 구현
- 선택한 배경 위에 일기 내용 올리기

💡 ChatGPT로 만든 이미지는 저작권 걱정 없이 앱에 사용할 수 있어요. '감사일기 앱 배경으로 쓸 노트 느낌의 이미지 만들어줘'라고 요청해보세요.



▲ ChatGPT로 만든 감사 카드 배경 이미지 3종

손글씨 폰트 적용하기

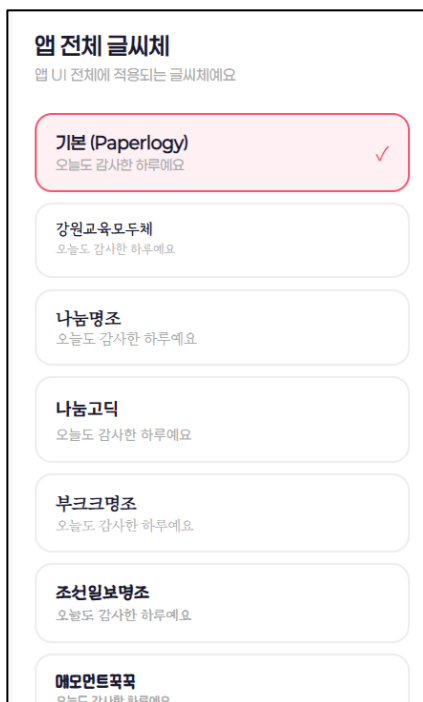
앱 전체에 손글씨 느낌의 폰트를 넣으면 훨씬 감성적으로 보여요. 저는 눈누(noonnu.cc)에서 무료 손글씨 폰트를 찾아서 적용했어요.

폰트 적용 방법은 간단해요. index.html 파일의 style 태그 안에 @font-face 코드를 추가하고, body의 font-family에 폰트 이름을 넣으면 돼요.

💡 눈누에서 '상업용 무료' 필터로 검색하면 앱에 사용해도 안전한 폰트만 나와요. 저작권 걱정 없이 쓸 수 있어요.

⚠ 폰트 사용 전 반드시 라이선스 확인을 하세요. '앱 광고 수익이 있어도 무료 사용 가능한지' 꼭 확인해야 해요. 글씨체 정보 상세페이지에 가보면 임베딩이 오케이인 걸로!.

처음엔 글씨체 하나만 넣으려고 했다가, 조금 더 다양한 느낌을 주고 싶어서 무료로 쓸 수 있는 글씨체를 여러 개 다운 받아서 입혀달라고 했어요



▲ 결국 13종의 글씨체를 입혔어요

앱 설정 페이지 링크 연결하기

앱의 설정 페이지에 이용약관, 개인정보처리방침, 문의하기 메뉴가 있다면 각각에 링크를 연결해야 해요.

- 이용약관 — 블로그나 구글 사이트에 간단히 작성 후 URL 연결
- 개인정보처리방침 — Play Store 등록 시 입력한 URL 과 동일하게 연결
- 문의하기 — 카카오톡 오픈채팅, 이메일, 또는 커뮤니티 링크

💡 `window.open('URL')` 코드로 버튼 클릭 시 브라우저가 열리도록 연결할 수 있어요.
Claude에게 '이 버튼 클릭하면 이 URL 열리게 해줘'라고 하면 돼요.



▲ 감사일기 설정 화면

AI 로 스티커 만들기

앱을 만들다 보면 욕심이 생겨요. "여기에 AI 기능 하나 넣으면 더 멋있지 않을까?" 저도 그랬어요. 사용자가 일기 내용을 입력하면, AI가 그 분위기에 맞는 스티커를 자동으로 만들어주는 기능. 멋있잖아요. 다른 일기와는 다른 차별점으로 너무 좋고 말예요.

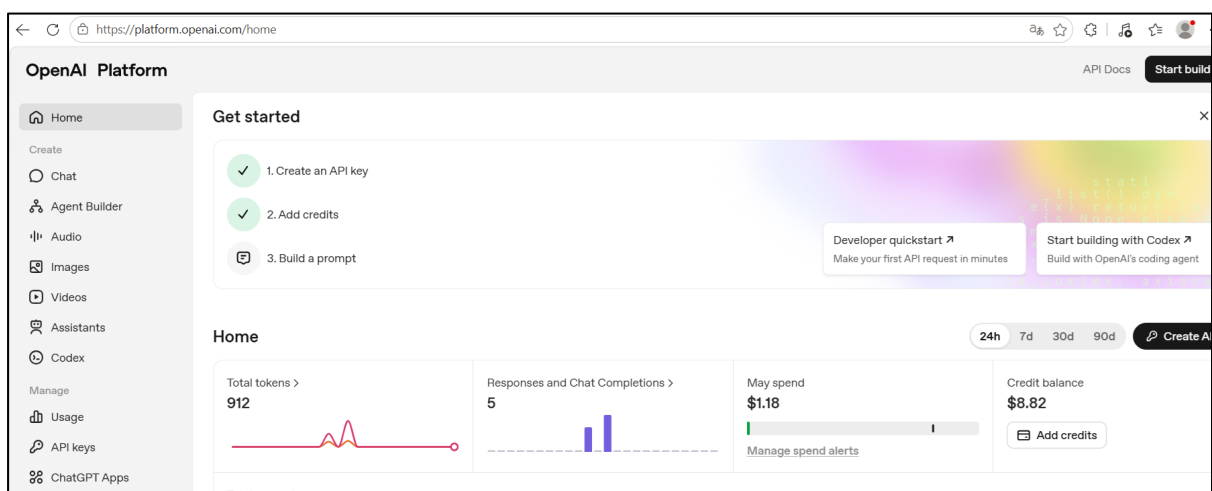
기획한 그림은 1) 사용자가 일기를 작성하고 2) AI 스티커 만들기 버튼을 클릭하면 3) AI가 일기 내용을 읽어서 어울리는 스티커 이미지를 자동 생성하는 거예요. 4) 일기 카드에 스티커를 추가할 수 있구요.

이걸 만들려면 OpenAI의 이미지 생성 API가 필요했어요. ChatGPT를 쓰는 곳과 OpenAI 개발자 페이지는 달라요.

API 키 발급 순서

- platform.openai.com 접속
- Google 계정으로 로그인
- 좌측 메뉴 "API Keys" 클릭
- "Create new secret key" 버튼
- 키 이름 입력 (예: "감사일기앱")
- 생성된 키 복사 → 이 키는 다시는 볼 수 없으니 안전한 곳에 저장!

△ API 키는 비밀번호와 같아요. 절대 누구한테도 알려주지 마시고, GitHub 같은 공개 저장소에 올리지 마세요. 누가 가져가면 그 사람이 내 계정으로 API를 막 써버릴 수 있어요.



▲ Open AI Platform 화면

비용 충전

OpenAI API는 사용한 만큼 돈이 빠져나가는 방식이에요. 미리 충전해두면 그 안에서 차감돼요. 한 사람이 이미지를 1장 만들 때마다 약 0.04 달러가 차감됩니다. 즉 50원 정도 차감되는 것이죠. 저는 처음에 10달러를 충전하고, 월 X회 무료 제공으로 해서 프리미엄 기능으로 수익화를 이뤄보려고 했어요.

하지만 수익화를 하기엔 뭔가 구독 결제 붙이는 것도 어렵고, 사업자도 필요하고 해서, 무료로 오픈하기로 했습니다.

💡 OpenAI Platform 들어가면 "Usage" 탭에서 얼마 썼는지 매일 확인할 수 있어요. 처음엔 자주 확인하세요. 코드 실수로 무한 호출되면 순식간에 돈 빠져요.

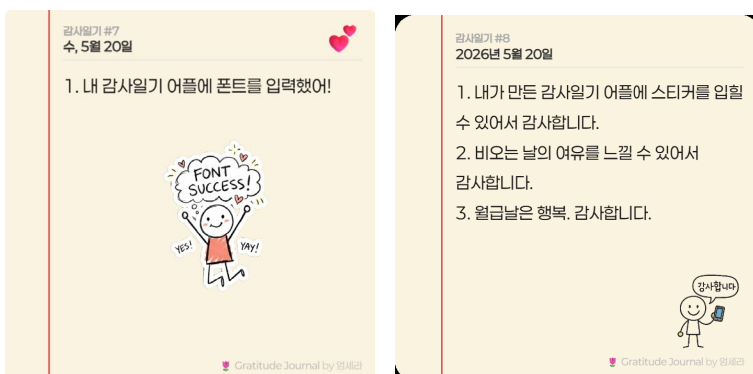
Claude 에게 연동 부탁하기

API 키를 받았으면 이제 앱이랑 연결해야 해요. Claude에게 그대로 부탁했어요.

"내 React 앱에 OpenAI API를 연동해줘. 사용자가 일기 텍스트를 입력하고 'AI 스티커 만들기' 버튼을 누르면, 그 텍스트에 어울리는 스티커 이미지가 생성되도록. API 키는 환경변수로 관리하고 싶어."

Claude가 코드를 다 짜줬어요. .env 파일에 API 키 저장하는 방법, 호출 함수, 생성된 이미지를 화면에 표시하는 코드까지요.

💡 API 키는 코드에 직접 쓰지 마세요. .env 파일에 따로 저장하고, .gitignore에 추가해서 Git에 안 올라가게 하세요. Claude에게 "API 키를 안전하게 관리하는 방법으로 코드 짜줘"라고 부탁하면 자동으로 처리해줘요.



▲ 내가 쓴 감사일기에 AI 스티커가 붙은 모습

처음 AI 스티커가 생성된 순간이 진짜 신기했어요. 일기에 "오늘 카페에서 책 읽었다"라고 쓰면, 따뜻한 분위기의 책과 커피 스티커가 자동으로 만들어졌거든요.

6 장. Google Play Store 에 올리기

사실 전 최소 기능만 갖춰서 일단 구글 플레이 스토어에 올려보기로 했어요. 완벽한 앱을 만들려고 욕심내다 보면 끝이 없거든요. '감사일기를 쓰고, 볼 수 있다'는 핵심 기능만 갖춰서 빠르게 출시하기로 마음먹었어요. 어차피 출시 후에도 계속 업데이트할 생각이였거든요.

그렇게 마음을 정하고 나니 진행이 빨라졌어요. 아이디어를 정하고 바이브코딩으로 만들기 시작해서, 비공개 테스트 트랙에 처음 올리기까지 3 주가 걸렸어요. (정식 출시는 거기서 또 시간이 더 걸렸는데, 그 이야기는 8 장에서 풀어드릴게요.)

그리고 또 하나 중요한 이유가 있었어요. 컴퓨터 브라우저에서 보는 거랑, 실제 제 핸드폰에서 사용해 보는 건 진짜 다르거든요. 컴퓨터에서 잘 보이던 글자가 핸드폰에선 잘려 보이기도 하고, 버튼을 누르려는데 손가락에 비해 너무 작기도 했어요. 비공개 테스트로 올려두면 제 핸드폰에 진짜 앱처럼 설치해서 일주일 동안 직접 써볼 수 있어요. 그러면서 진짜 고쳐야 할 부분이 보이더라고요.

💡 비코더가 가장 자주 하는 실수: "완벽하게 만들고 출시해야지." 이러다 영원히 출시 못 해요. 핵심 기능만 갖춰서 일단 올리고, 사용자 반응 보면서 개선하세요. 이게 MVP(Minimum Viable Product, 최소 기능 제품) 정신이에요.

Play Store 출시 전 준비물 체크리스트

- 서명된 AAB 파일 — Android Studio 에서 Build > Generate Signed Bundle 로 만들어요
- Keystore 파일 — 앱 서명에 필요한 파일이에요. 잃어버리면 안 돼요!
- 앱 아이콘 — 512x512 PNG 파일
- 그래픽 이미지 — 1024x500 PNG 파일. Play Store 상단에 보이는 배너예요.
- 스크린샷 — 최소 2 장, 최대 8 장. 앱의 주요 화면을 캡처해요.
- 앱 이름 — 30 자 이내
- 간단한 설명 — 80 자 이내
- 자세한 설명 — 4000 자 이내
- 개인정보처리방침 URL — 블로그나 웹사이트에 올려두면 돼요

⚠ Keystore 파일은 앱의 서명 키예요. 한 번 만들면 계속 써야 하고, 잃어버리면 같은 앱 이름으로 업데이트를 올릴 수 없어요. 반드시 외장하드, 클라우드 등 여러 곳에 백업해두

세요!

AAB 빌드하는 전체 순서

코드 수정이 완료됐다면 Play Store에 올릴 파일을 만드는 순서예요.

- 1 단계: npm run build (VS Code 터미널)
- 2 단계: npx cap sync (VS Code 터미널)
- 3 단계: npx cap open android (Android Studio 열기)
- 4 단계: Build > Generate Signed Bundle / APK
- 5 단계: Android App Bundle 선택 > Next
- 6 단계: Keystore 파일 선택, 비밀번호 입력
- 7 단계: release 선택 > Create
- 8 단계: 생성된 app-release.aab 파일을 Play Console 에 업로드

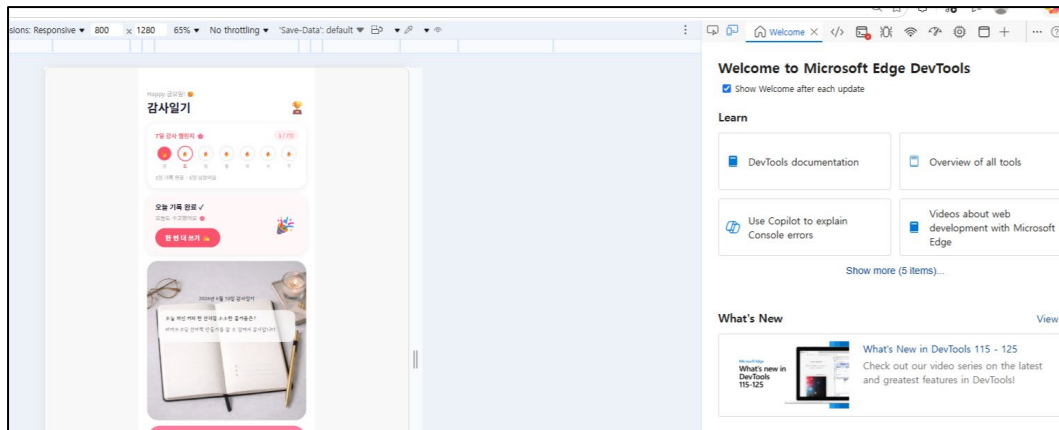
💡 *Build Variants에서 'release'로 변경한 후 Generate Bundle을 하면 서명된 release 버전이 만들어져요. debug 버전은 Play Store에 올릴 수 없어요. 여기서 무슨 말인지 모르시겠죠? AI가 순서대로 알려주니 그대로 따라하시면 됩니다.*

스크린샷 촬영 팁

스크린샷은 앱의 첫인상을 결정해요. 예쁘고 명확한 스크린샷이 다운로드 수에 큰 영향을 미쳐요.

브라우저에서 F12를 누르고 Ctrl+Shift+M으로 모바일 뷰로 전환해서 캡처하면 실제 핸드폰 화면처럼 보여요. 크기는 390x844 정도로 설정하면 깔끔해요.

💡 *스크린샷에는 앱의 가장 매력적인 화면을 보여주세요. 홈 화면, 일기 작성 화면, 완성된 일기 화면 등 다양하게 넣으면 좋아요.*



▲ 스크린샷 촬영 시 사이즈 변경하는 모습

비공개 테스트는 왜 해야 할까

처음 Play Console에 가입한 개발자는 바로 정식 출시를 할 수 없어요. 비공개 테스트를 먼저 진행해야 해요.

Google에서는 12명의 테스터가 14일 이상 앱을 사용해야 프로덕션(정식 출시) 액세스 권한을 줘요. 테스터는 주변 친구들이나 SNS 팔로워에게 부탁하면 돼요. 전 지인에게 요청하거나 제가 속한 온라인 커뮤니티 방에 요청했다가 원하는 만큼 안 나와서, 결국 크몽(kmong.com)에서 유료 테스터를 의뢰했어요. 검색창에 "구글 플레이 테스터", "앱 테스터 14일" 같은 키워드 검색하면 나와요.

💡 직접 테스터를 구할 시, 테스터에게 구체적인 안내를 해주세요. '이 링크로 들어가서 테스터 신청하고, 앱 설치해서 한 번만 써봐줘'라고 하면 돼요.

💡 크몽에서 의뢰할 시, 저는 29,000원이었는데 안내해준 대로 진행하니까 훨씬 더 편하더라고요

7 장. 출시 이후 — 앱을 성장시키는 방법

출시가 끝이 아니에요, 진짜 시작이에요

앱을 출시하면 모든 게 끝난 것 같지만, 사실 이제부터가 진짜 시작이에요. 출시 후 꾸준한 업데이트와 마케팅이 앱의 성공을 결정해요.

사용자 피드백 듣기

출시 후 리뷰를 주의 깊게 읽으세요. 사용자들이 무엇을 원하는지, 어떤 부분이 불편한지 알 수 있어요.

💡 나쁜 리뷰도 달게 받아들이세요. 욕을 하는 리뷰는 무시하더라도, 불편함을 이야기하는 리뷰는 소중한 개선 힌트예요.

계속 업데이트하기

바이브코딩의 장점이 여기서 빛을 발해요. 새 기능을 추가하고 싶으면 AI에게 '이런 기능 추가해줘'라고 하면 되거든요. 제 다음 업데이트 내용이에요.

- 새소리, 명상음악 등 배경음악 추가
- 감사 일기 인사이트 분석
- 유료 결제 기능 추가

14 일 후 – 프로덕션 액세스 신청

저는 크몽에 비공개 테스트를 유료로 의뢰하는 대신 기본 UI/UX를 업데이트 하는 데 집중했어요. 14일이 다 차니 Play Console에 알림이 뜨더라구요. “이제 프로덕션을 신청할 수 있습니다”

8 장. 첫 출시 거부 – 진짜 시작은 여기서부터

14일의 비공개 테스트 기간이 끝나고 프로덕션까지 신청해서, 이젠 진짜 앱을 출시할 수 있을 줄 알았어요. 그런데 며칠 지나지 않아 프로덕션이 거부되었다는 메일을 받게 되었습니다.

발견된 문제: 잘못된 개인정보처리방침

개인정보처리방침에 다음과 같은 문제가 있습니다.

1. 개인정보처리방침 링크가 요건을 충족하지 않음

- URL이 활성화되어 있고, 수정하거나 댓글을 달 수 없으며, PDF로 연결되지 않고, 비밀번호로 보호되지 않으며, 전 세계 어디서나 공개적으로 액세스할 수 있고, 파일을 자동으로 다운로드하지 않는지 확인합니다.

2. 데이터 보관 방법을 명시하지 않음

- 사용자 데이터를 저장/보관하는 기간을 사용자에게 알려야 합니다.

3. 데이터 삭제 방법을 명시하지 않음

- 사용자의 데이터가 저장/보관되는 기간과 사용자가 앱에서 데이터를 삭제할 수 있는 방법을 사용자에게 알려야 합니다.

세 가지 사유가 모두 개인정보처리방침 관련이었어요. 처음엔 ‘이게 무슨 소리인가?’ 싶어서 또 클로드에게 물어보고 해결방안을 같이 얘기했죠.

문제점은 명확했습니다. 1) 개인정보처리방침이 너무 간단했고, 2) 정책과 실제 앱이 불일치 (애드몹 광고 관련), 3) 데이터 보관/삭제 방법이 누락되어 있었어요. 부족한 부분을 수정하면서 다른 버그는 없는지 한번 또 찾아서 보완했습니다.

거부가 가르쳐준 것

거부받기 전에는 "출시 = 끝"이라고 생각했어요. 거부받고 나서 알았어요. 출시는 끝이 아니었다는 것을. 거부는 손해가 아니라 점검 기회였어요. 바이브코딩으로 어플을 만들었듯이, 거부가 났을 때도 똑같이 바이브코딩으로 다시 승인될 수 있게 셋팅하면 되는 것이죠.

그리고 또 하나 알았어요. 거부는 콘텐츠가 된다는 것. 만약 첫 시도에 바로 출시가 됐다면, 저는 "출시 완료!"라는 글 하나만 쓰고 끝났을 거예요. 근데 거부 받고 주말 동안 고생한 이야기가 더 풍부한 콘텐츠가 되더라구요.

에러는 해결할 수 있어요.

- 작게 시작하세요. 처음부터 복잡한 앱을 만들려고 하면 지쳐요.
- 기록하세요. 어떤 에러가 났고 어떻게 해결했는지 적어두면 나중에 도움이 돼요.
- 커뮤니티를 찾으세요. 같은 목표를 가진 사람들과 함께하면 훨씬 힘이 돼요.

여러분의 머릿속에 있는 아이디어는 세상에 필요한 것일 수 있어요. 그 아이디어를 실현하는데 더 이상 코딩 실력이 장벽이 되지 않아도 되는 시대가 왔어요.

당신도 할 수 있어요. 정말로. 🍀

— 엄세라

부록 — 자주 묻는 질문 (FAQ)

Q. 맥북이 없으면 안 되나요?

윈도우로도 충분히 가능해요. 저도 윈도우로 했어요. 단, iOS 앱을 만들려면 맥북이 필요해요. Android 앱만 만들 거라면 윈도우로도 OK예요.

Q. 코딩을 전혀 몰라도 되나요?

네, 전혀 몰라도 돼요. 하지만 기본 개념들(파일, 폴더, 터미널 등)에 조금씩 익숙해지면 훨씬 수월해요.

Q. 얼마나 걸리나요?

개인차가 있지만, 저는 처음 시작해서 Play Store 출시까지 약 3주 걸렸어요. 매일 조금씩 했고, 막히는 부분에서 시간이 많이 걸렸어요.

Q. 비용이 얼마나 드나요?

Google Play 개발자 등록비 25달러(약 3만8천원)가 들었고, Claude 유료 구독으로 따지면 꽤 크게 들었네요. (연간 30만원) 물론 AI 도구는 무료 버전도 사용할 수 있지만, 전 참을성이 없는 편이라 바로 유료구독을 해버렸어요. Android Studio, VS Code는 모두 무료예요.

Q. 앱이 승인 거부되면 어떡하나요?

걱정하지 마세요. 거부 사유를 확인하고 수정하면 재심사를 받을 수 있어요. 대부분의 거부는 정책 관련 내용이라 수정하면 통과돼요.

Q. AI가 짠 코드를 내가 이해해야 하나요?

처음엔 이해 못 해도 괜찮아요. 하지만 조금씩 읽다 보면 어느 순간 패턴이 보이기 시작해요. 완전히 이해하지 못해도 앱을 만들 수 있어요.

Q. 새 기능을 추가할 때마다 AAB를 다시 빌드해야 하나요?

네, 앱을 업데이트할 때마다 `npm run build` → `npx cap sync` → Android Studio에서 AAB 빌드 순서로 진행해야 해요. 처음엔 번거롭지만 금방 익숙해져요.

Q. 태블릿 스크린샷도 꼭 올려야 하나요?

Play Store 등록 시 태블릿 스크린샷도 요구해요. 핸드폰 스크린샷과 다른 화면을 올리거나, 브라우저에서 800x1280 크기로 캡처하면 돼요.